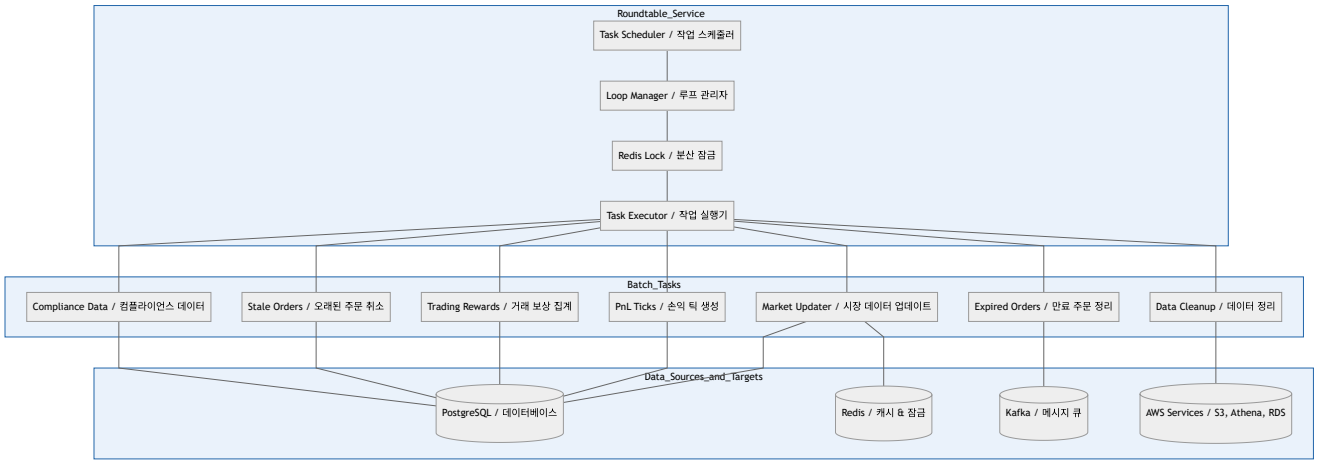
Roundtable 배치 작업 서비스 구현 분석

개요

Roundtable은 dYdX v4 Indexer의 작업 스케줄링 시스템으로, 다양한 주기적 배치 작업들을 관리하고 실행하는 서비스입니다. 크론잡과 유사한 기능을 제공하며, 시장 데이터 업데이트, 만료된 주문 정리, PnL 틱 생성, 거래 보상 집계 등의 중요한 백그라운드 작업들을 담당합니다.

아키텍처 개요



주기적 작업 스케줄링 시스템

1. 작업 스케줄러 초기화

```
async function start(): Promise<void> {

startBugsnag();
```

```
await Promise.all([
producer.connect(),
connectToRedis(),
]);

// 각 작업별 조건부 시작
if (config.LOOPS_ENABLED_MARKET_UPDATER) {
startLoop(
marketUpdaterTask,
'market_updater',
config.LOOPS_INTERVAL_MS_MARKET_UPDATER,
config.MARKET_UPDATER_LOCK_MULTIPLIER,
);
}

if (config.LOOPS_ENABLED_PNL_TICKS) {
startLoop(
createPnlTicksTask,
'create_pnl_ticks',
config.LOOPS_INTERVAL_MS_PNL_TICKS,
config.PNL_TICK_UPDATE_LOCK_MULTIPLIER,
);
```

```
}

// 거래 보상 집계 작업들 (일간, 주간, 월간)
if (config.LOOPS_ENABLED_AGGREGATE_TRADING_REWARDS_DAILY) {
startLoop(
aggregateTradingRewardsTasks(TradingRewardAggregationPeriod.DAILY),
'aggregate_trading_rewards_daily',
config.LOOPS_INTERVAL_MS_AGGREGATE_TRADING_REWARDS,
);
}

// 기타 작업들...
}
```

2. 루프 관리 시스템

```
export function startLoop(
loopTask: () => Promise<unknown>,
taskName: string,
loopIntervalMs: number,
extendedLoopLockMultiplier?: number,
): void {
wrapBackgroundTask(
```

```
startLoopAsync(loopTask, taskName, loopIntervalMs, extendedLoopLockMultiplier),
true,
'taskName',
);
}

async function startLoopAsync(
loopTask: () => Promise<unknown>,
taskName: string,
loopIntervalMs: number,
extendedLoopLockMultiplier?: number,
): Promise<void> {
// 시작 지연 (여러 인스턴스 간 부하 분산)
if (config.START_DELAY_ENABLED) {
await delay(generateRandomStartDelayMs(loopIntervalMs));
}

for (;;) {
// 동시 실행 작업 수 제한
if (numRunningTasks > config.MAX_CONCURRENT_RUNNING_TASKS) {
stats.increment(exceededMaxConcurrentTasksStat, { taskName });
await delay(config.EXCEEDED_MAX_CONCURRENT_RUNNING_TASKS_DELAY_MS);
continue;
```

```
}

// Redis 분산 잠금 획득
const lockResult: boolean = await redis.lockWithExpiry(
redisClient,
redisKey,
REDIS_VALUE,
loopIntervalMs,
);

if (!lockResult) {
await waitUntilKeyExpiresPlusJitter(redisKey);
continue;
}

// 확장 잠금 (장시간 실행 작업용)
let extendedLockResult: boolean = true;
const extendedLockRedisValue: string = `${REDIS_VALUE} ${uuidv4()}`;
if (extendedLoopLockMultiplier !== undefined) {
extendedLockResult = await redis.lockWithExpiry(
redisClient,
extendedLockKey,
extendedLockRedisValue,
```

```
loopIntervalMs * extendedLoopLockMultiplier,
);
}

if (!extendedLockResult) {
await redis.unlock(redisClient, redisKey, REDIS_VALUE);
await waitUntilKeyExpiresPlusJitter(extendedLockKey);
continue;
}

// 작업 실행
const start: number = Date.now();
numRunningTasks += 1;

try {
await loopTask();
stats.gauge(completedStat, 1);
} catch (error) {
stats.gauge(completedStat, 0);
logger.error({
message: 'uncaught error in an individual loop',
error,
taskName,
```

```
});
} finally {
numRunningTasks -= 1;
// 확장 잠금 해제
if (extendedLoopLockMultiplier !== undefined && extendedLockResult) {
await redis.unlock(redisClient, extendedLockKey, extendedLockRedisValue);
}

// 성능 메트릭 수집
const loopDuration: number = Date.now() - start;
stats.timing(timingStat, loopDuration);
stats.gauge(durationRatioStat, loopDuration / loopIntervalMs, { taskName });
}

// 다음 실행까지 대기
await waitUntilKeyExpiresPlusJitter(redisKey);
}
}
```#
3. 분산 잠금 메커니즘

```typescript
// 기본 잠금 + 확장 잠금 시스템
```

```
const lockResult: boolean = await redis.lockWithExpiry(
redisClient,
redisKey, // 기본 잠금 키
REDIS_VALUE, // 잠금 값
loopIntervalMs, // 기본 만료 시간
);


// 장시간 실행 작업을 위한 확장 잠금
const extendedLockResult: boolean = await redis.lockWithExpiry(
redisClient,
extendedLockKey, // 확장 잠금 키
extendedLockRedisValue, // 고유 잠금 값 (UUID 포함)
loopIntervalMs * extendedLoopLockMultiplier, // 확장된 만료 시간
);


// 지터를 포함한 대기 시간 계산
async function waitUntilKeyExpiresPlusJitter(redisKey: string): Promise<void>
const pttl: number = await redis.pttl(redisClient, redisKey);
const jitter: number = Math.ceil(Math.random() * (pttl * config.JITTER_FRACTION
if (pttl > 0) {
await delay(pttl + jitter);
}
```

```
}
```

캔들스틱 데이터 생성 알고리즘

1. 시장 데이터 업데이트 작업

```
export default async function runTask(): Promise<void> {
// 기본 데이터 조회
const [liquidityTiers, perpetualMarkets] = await Promise.all([
LiquidityTiersTable.findAll({}, []),
PerpetualMarketTable.findAll({}, []),
]);

const perpetualMarketIds: string[] = _.map(perpetualMarkets, PerpetualMarketCol
const clobPairIds: string[] = _.map(perpetualMarkets, PerpetualMarketColumns.cl
const tickers: string[] = _.map(perpetualMarkets, PerpetualMarketColumns.ticker

// 가격 데이터 조회
const [latestPrices, prices24hAgo] = await Promise.all([
OraclePriceTable.getLatestPrices(),
OraclePriceTable.getPricesFrom24hAgo(),
]);
```

```
// 시장 통계 데이터 수집

const [perpetualMarketStats, openInterest, fundingRates] = await  Promise.all([

FillTable.get24HourInformation(clobPairIds), // 24시간 거래량/횟수

PerpetualPositionTable.getOpenInterestLong(perpetualMarketIds), // 미결제약정

NextFundingCache.getNextFunding(redisClient, tickers), // 펀딩 레이트

]);



// 가격 변화율 계산

const  priceChange24H = getPriceChange(marketId, latestPrices, prices24hAgo);



// 데이터베이스 업데이트

const  txId: number = await  Transaction.start();

try {

await  PerpetualMarketTable.updateMarketCheckerFields(

perpetualMarkets.map((perpetualMarket) => ({

id:  perpetualMarket.id,

trades24H:  parseInt(perpetualMarketStats[perpetualMarket.id].trades24H, 10) ??

volume24H:  perpetualMarketStats[perpetualMarket.id].volume24H ?? '0',

priceChange24H:  priceChange24H ?? perpetualMarket.priceChange24H,

openInterest:  openInterest[perpetualMarket.id]?.openInterest ?? '0',

nextFundingRate:  fundingRates[perpetualMarket.ticker]?.toFixed() ?? perpetualMa

})),

Transaction.get(txId),
```

```
);
await Transaction.commit(txId);
} catch (error) {
await Transaction.rollback(txId);
throw error;
}

// WebSocket 메시지 생성 및 전송
compareAndHandleMarketsWebsocketMessage({
oldMarkets: perpetualMarketMap,
newMarkets: updatedMarketsMap,
liquidityTiers: liquidityTiersMap,
});
}

export function getPriceChange(
marketId: number,
latestPrices: PriceMap,
prices24hAgo: PriceMap,
): string | undefined {
const latestPrice: string | undefined = latestPrices[marketId];
const price24hAgo: string | undefined = prices24hAgo[marketId];
if (latestPrice === undefined || price24hAgo === undefined) {
```

```
return  undefined;

}

return new Big(latestPrice).minus(price24hAgo).toFixed();

}
```

2. PnL 틱 생성 알고리즘

```
export default async function runTask(): Promise<void> {

// 최신 블록과 마지막 처리된 블록 시간 조회

const [block, pnlTickLatestBlocktime] = await Promise.all([

BlockTable.getLatest({ readReplica: true }),

PnlTicksTable.findLatestProcessedBlocktime(),

]);


const latestBlockTime: string = block.time;

const latestBlockHeight: string = block.blockHeight;


// 업데이트 간격 확인

if (Date.parse(latestBlockTime) - normalizeStartTime(new Date(pnlTickLatestBlock

< config.PNL_TICK_UPDATE_INTERVAL_MS) {

logger.info({

message: 'Skipping run because update interval has not been reached',

pnlTickLatestBlocktime,
```

```
latestBlockTime,
threshold: config.PNL_TICK_UPDATE_INTERVAL_MS,
});
return;
}


// 트랜잭션 내에서 일관된 데이터 읽기
const txId: number = await Transaction.start();
let newTicksToCreate: PnlTicksCreateObject[] = [];
try {
newTicksToCreate = await getPnlTicksCreateObjects(latestBlockHeight, latestBlock
} finally {
await Transaction.rollback(txId); // 읽기 전용이므로 롤백
}


// 배치로 새 틱 생성
const newTicks: PnlTicksCreateObject[] = await batchCreateNewTicks(newTicksToCre
// Redis 캐시 업데이트
const newestTicksPerSubaccount = _.keyBy(newTicks, 'subaccountId');
await LatestAccountPnlTicksCache.set(newestTicksPerSubaccount, redisClient);
}


async function batchCreateNewTicks(ticks: PnlTicksCreateObject[]): Promise<PnlTi
```

```
const newTicks: PnlTicksCreateObject[] = [];

// 배치 크기로 청크 분할

const chunkedTicks = _.chunk(ticks, config.PNL_TICK_MAX_ROWS_PER_UPSERT);

for (const ticksToCreate of chunkedTicks) {

const txId: number = await Transaction.start();

try {

await PnlTicksTable.createMany(ticksToCreate, { txId });

await Transaction.commit(txId);

} catch (e) {

await Transaction.rollback(txId);

throw e;

}

newTicks.push(...ticksToCreate);

}



stats.gauge(`${config.SERVICE_NAME}_pnl_tick_create_count`, newTicks.length);

return newTicks;

}
```

통계 데이터 계산 및 집계 로직

1. 거래 보상 집계 시스템

```
export class AggregateTradingReward {
period: TradingRewardAggregationPeriod;

async runTask(): Promise<void> {
// 불완전한 집계 데이터 정리
await this.maybeDeleteIncompleteAggregatedTradingReward();
// 처리할 시간 간격 계산
const interval: Interval = await this.getTradingRewardDataToProcessInterval();
// 간격별 거래 보상 데이터 수집
const intervalTradingRewardsByAddress = await this.getIntervalTradingRewardsByAd
// 집계 데이터 업데이트
await this.updateTradingRewardsAggregation(interval, intervalTradingRewardsByAdd
// 처리 완료 시간 기록
await this.setProcessedTime(interval.end.toISO());
}

private async getIntervalTradingRewardsByAddress(
interval: Interval,
): Promise<IntervalTradingRewardsByAddress> {
// 시간 간격 내 거래 보상 조회
const tradingRewards: TradingRewardFromDatabase[] = await TradingRewardTable.fin
blockTimeAfterOrAt: interval.start.toISO(),
```

```
blockTimeBefore: interval.end.toISO(),

}, []);



// 주소별 그룹화

const tradingRewardsByAddress = _.groupBy(tradingRewards, TradingRewardColumns.a



// 주소별 보상 합계 계산

return _.mapValues(tradingRewardsByAddress, (tradingRewardsForAddress) => {

return _.reduce(tradingRewardsForAddress, (sum: string, tradingReward) => {

return Big(sum).plus(tradingReward.amount).toFixed();

}, '0');

});

}



private async updateTradingRewardsAggregation(

interval: Interval,

intervalTradingRewardsByAddress: IntervalTradingRewardsByAddress,

): Promise<void> {

let aggregationUpdateAndCreateObjects =

await this.getAggregationUpdateAndCreateObjectsFromInterval(

interval,

intervalTradingRewardsByAddress

);
```

```
// 기간 종료 시 endedAt 및 endedAtHeight 설정

if (this.isEndofPeriod(interval.end)) {

aggregationUpdateAndCreateObjects =
await this.addEndedAtAndEndedAtHeightUpdates(

aggregationUpdateAndCreateObjects,

interval,

);

}



// 트랜잭션으로 업데이트 및 생성 실행

const txId: number = await Transaction.start();

await Transaction.setIsolationLevel(
txId, IsolationLevel.READ_UNCOMMITTED);

try {

await this.createAndUpdateAggregations(
aggregationUpdateAndCreateObjects, txId);

await Transaction.commit(txId);

} catch (error) {

await Transaction.rollback(txId);

throw error;

}

}



private generateInterval(startTime: DateTime, latestBlock: BlockFromDatabase): I
```

```
const latestBlockTime: Date = DateTime.fromISO(latestBlock.time, UTC_OPTIONS).to

const normalizedLatestBlockTime: Date = floorDate(latestBlockTime, ONE_MINUTE_IN

const nextDay: Date = startTime.plus({ days: 1 }).toJSDate();

const normalizedNextDay: Date = floorDate(nextDay, ONE_DAY_IN_MILLISECONDS);

const startTimePlusMaxIntervalSize: Date = DateTime.fromJSDate(startTime.toJSDat

.plus({ milliseconds: config.AGGREGATE_TRADING_REWARDS_MAX_INTERVAL_SIZE_MS }).t

// 가장 이른 시간을 종료 시간으로 선택

const endTime: Date = new Date(Math.min(

normalizedLatestBlockTime.getTime(),

normalizedNextDay.getTime(),

startTimePlusMaxIntervalSize.getTime(),

));

return Interval.fromDateTimes(startTime, DateTime.fromJSDate(endTime).toUTC());

}

}

```## 데

이터 정리 및 아카이브 프로세스

1. 만료된 주문 제거 작업


```typescript

export default async function runTask(): Promise<void> {

const block: BlockFromDatabase =
```

```
    await BlockTable.getLatest({ readReplica: true });



    try {

    // 단기 주문만 만료 처리 (장기 주문은 온체인에서 처리)

    const expiryCutoff: number = (

    +block.blockHeight -
    config.BLOCKS_TO_DELAY_EXPIRY_BEFORE_SENDING_REMOVES

    );

    // 만료된 주문 UUID 조회

    const orderUuids: string[] =
    await OrderExpiryCache.getOrderExpiries({

    latestExpiry: expiryCutoff,

    latestExpiryIsInclusive: true,

    }, redisClient) as string[];



    // Kafka 배치 프로듀서 생성

    const batchKafkaProducer: BatchKafkaProducer =
    new BatchKafkaProducer(

    KafkaTopics.TO_VULCAN,

    producer,

    config.KAFKA_MAX_BATCH_WEBSOCKET_MESSAGE_SIZE_BYTES,

    );



    // 각 주문에 대해 만료 메시지 생성

    const allPromises: Promise<void>[] = _.map(
```

```
orderUuids, (orderUuid: string) => {

return addExpireOrderMessageOrFail(
orderUuid, expiryCutoff, batchKafkaProducer);

});



// 모든 메시지 추가 완료 후 배치 전송

await Promise.allSettled(allPromises);

await batchKafkaProducer.flush();

} catch (error) {

logger.error({

message: 'Error occurred in task to remove expired orders',

error,

});

}

}



async function addExpireOrderMessageOrFail(

orderUuid: string,

expiryCutoff: number,

messenger: BatchKafkaProducer,

): Promise<void> {

try {

// Redis에서 주문 데이터 조회

const [redisOrder, orderData] = await Promise.all([
```

```
OrdersCache.getOrder(orderUuid, redisClient),

OrdersDataCache.getOrderDataWithUUID(orderUuid, redisClient),

]);



if (redisOrder == null || orderData == null) {

stats.increment(`${config.SERVICE_NAME}
.expired_order_data_not_found`, 1);

return;

}



const order: IndexerOrder = redisOrder.order!;

// 만료 시간 검증

if (order.goodTilBlock! > expiryCutoff) {

stats.increment(`${config.SERVICE_NAME}
.indexer_expired_order_has_newer_expiry`, 1);

return;

}



// 완전 체결된 주문 통계

if (order.quantums.lte(orderData.totalFilledQuantums)) {

stats.increment(`${config.SERVICE_NAME}
.fully_filled_orders_expired_by_roundtable`, 1);

}
```

```
// 만료 메시지 추가

messenger.addMessageAndMaybeFlush({

key: getOrderIdHash(order.orderId!),

value: getExpiredOffChainUpdateMessage(order.orderId!),

});



stats.increment(`${config.SERVICE_NAME}.expiry_message_sent`, 1);

} catch (error) {

logger.error({

message: 'Encountered error expiring order',

orderUuid,

expiryCutoff,

error,

});

}

}
```

2. 오래된 주문 취소 작업

```
export default async function runTask(): Promise<void> {

const latestBlock: BlockFromDatabase = await BlockTable.getLatest({ readReplica:

const latestBlockHeight: number = parseInt(latestBlock.blockHeight, 10);
```

```
// 오래된 OPEN 상태의 단기 주문 조회
const staleOpenOrders: OrderFromDatabase[] = await OrderTable.findAll({
statuses: [OrderStatus.OPEN],
orderFlags: ORDER_FLAG_SHORT_TERM.toString(),
// goodTilBlock이 현재 블록 높이보다 작은 주문들
goodTilBlockBeforeOrAt: (latestBlockHeight - 1).toString(),
limit: config.CANCEL_STALE_ORDERS_QUERY_BATCH_SIZE,
}, [], { readReplica: true });


// 주문 상태를 CANCELED로 업데이트
const orderIds: string[] = staleOpenOrders.map((order) => order.id);
const canceledOrders: OrderFromDatabase[] = await OrderTable.updateStaleOrderSta
OrderStatus.OPEN,
OrderStatus.CANCELED,
latestBlock.blockHeight,
orderIds,
);


// 통계 수집
stats.gauge(`${config.SERVICE_NAME}.num_stale_orders.count`, orderIds.length);
stats.gauge(`${config.SERVICE_NAME}.num_stale_orders_canceled.count`, canceledOr
}
```

3. 데이터 아카이브 및 스냅샷 작업

```
// Fast Sync 스냅샷 생성
export default async function runTask(): Promise<void> {
try {
// RDS 스냅샷 생성
const snapshotIdentifier = `${config.FAST_SYNC_SNAPSHOT_IDENTIFIER_PREFIX}-${Dat
await createDBSnapshot({
DBInstanceIdentifier: config.RDS_INSTANCE_NAME,
DBSnapshotIdentifier: snapshotIdentifier,
});

// S3에 메타데이터 저장
await uploadSnapshotMetadata(snapshotIdentifier);
logger.info({
message: 'Fast sync snapshot created successfully',
snapshotIdentifier,
});
} catch (error) {
logger.error({
message: 'Failed to create fast sync snapshot',
error,
});
```

```
}

}



// 오래된 스냅샷 삭제

export default async function runTask(): Promise<void> {

try {

const cutoffDate = new Date(Date.now() - config.SNAPSHOT_RETENTION_DAYS * 24 * 6

// 오래된 스냅샷 목록 조회

const oldSnapshots = await listOldSnapshots(cutoffDate);

// 배치로 삭제

for (const snapshot of oldSnapshots) {

await deleteDBSnapshot(snapshot.DBSnapshotIdentifier);

logger.info({

message: 'Deleted old snapshot',

snapshotIdentifier: snapshot.DBSnapshotIdentifier,

});

}

} catch (error) {

logger.error({

message: 'Failed to delete old snapshots',

error,

});

}
```

```
}

```### 4

. 컴플라이언스 데이터 관리


```typescript

export default async function runTask(): Promise<void> {

try {

// 활성 주소 임계값 계산

const activeThresholdTime = new Date(

Date.now() - config.ACTIVE_ADDRESS_THRESHOLD_SECONDS * 1000

);



// 업데이트가 필요한 주소들 조회

const addressesToUpdate = await getAddressesNeedingComplianceUpdate(activeThresh

// 배치 크기로 분할하여 처리

const batches = _.chunk(addressesToUpdate, config.COMPLIANCE_PROVIDER_QUERY_BATC

for (const batch of batches) {

// 외부 컴플라이언스 제공업체에서 데이터 조회

const complianceData = await complianceProvider.queryBatch(batch);

// 데이터베이스 업데이트

await updateComplianceDataInDatabase(complianceData);

// API 호출 간 지연

await delay(config.COMPLIANCE_PROVIDER_QUERY_DELAY_MS);
```

```
}


// 상태 전환 처리 (CLOSE_ONLY -> BLOCKED)
await performComplianceStatusTransitions();
} catch (error) {
logger.error({
message: 'Error in compliance data update task',
error,
});
}
}


async function performComplianceStatusTransitions(): Promise<void> {
const cutoffDate = new Date(
Date.now() - config.CLOSE_ONLY_TO_BLOCKED_DAYS * 24 * 60 * 60 * 1000
);


// CLOSE_ONLY 상태가 일정 기간 지속된 주소들을 BLOCKED로 전환
const addressesToBlock = await ComplianceTable.findAll({
status: ComplianceStatus.CLOSE_ONLY,
updatedAtBefore: cutoffDate.toISOString(),
});
```

```
for (const address of addressesToBlock) {
await ComplianceTable.update({
address: address.address,
status: ComplianceStatus.BLOCKED,
reason: 'Automatic transition from CLOSE_ONLY after grace period',
});
}
}
```

작업별 설정 및 스케줄링

1. 작업 설정 구조

```
export const configSchema = {
// 작업 활성화 플래그
LOOPS_ENABLED_MARKET_UPDATER: parseBoolean({ default: true }),
LOOPS_ENABLED_DELETE_ZERO_PRICE_LEVELS: parseBoolean({ default: true }),
LOOPS_ENABLED_PNL_TICKS: parseBoolean({ default: true }),
LOOPS_ENABLED_REMOVE_EXPIRED_ORDERS: parseBoolean({ default: true }),
LOOPS_ORDERBOOK_INSTRUMENTATION: parseBoolean({ default: true }),
LOOPS_CANCEL_STALE_ORDERS: parseBoolean({ default: true }),
LOOPS_ENABLED_AGGREGATE_TRADING_REWARDS_DAILY: parseBoolean({ default: true }),
LOOPS_ENABLED_AGGREGATE_TRADING_REWARDS_WEEKLY: parseBoolean({ default: true }),
```

```
LOOPS_ENABLED_AGGREGATE_TRADING_REWARDS_MONTHLY: parseBoolean({ default: true })


// 작업 실행 간격
LOOPS_INTERVAL_MS_MARKET_UPDATER: parseInteger({
default: THIRTY_SECONDS_IN_MILLISECONDS,
}),
LOOPS_INTERVAL_MS_DELETE_ZERO_PRICE_LEVELS: parseInteger({
default: 2 * ONE_MINUTE_IN_MILLISECONDS,
}),
LOOPS_INTERVAL_MS_PNL_TICKS: parseInteger({
default: THIRTY_SECONDS_IN_MILLISECONDS,
}),
LOOPS_INTERVAL_MS_REMOVE_EXPIRED_ORDERS: parseInteger({
default: 2 * ONE_MINUTE_IN_MILLISECONDS,
}),
LOOPS_INTERVAL_MS_AGGREGATE_TRADING_REWARDS: parseInteger({
default: THIRTY_SECONDS_IN_MILLISECONDS,
}),


// 시작 지연 설정
START_DELAY_ENABLED: parseBoolean({ default: true }),
MAX_START_DELAY_MS: parseInteger({ default: 3 * ONE_MINUTE_IN_MILLISECONDS }),
MAX_START_DELAY_FRACTION_OF_INTERVAL: parseNumber({ default: 0.1 }),
```

```
JITTER_FRACTION_OF_DELAY: parseNumber({ default: 0.01 }),


// 잠금 승수 (장시간 실행 작업용)
MARKET_UPDATER_LOCK_MULTIPLIER: parseInteger({ default: 10 }),
DELETE_ZERO_PRICE_LEVELS_LOCK_MULTIPLIER: parseInteger({ default: 1 }),
PNL_TICK_UPDATE_LOCK_MULTIPLIER: parseInteger({ default: 20 }),


// 동시 실행 제한
MAX_CONCURRENT_RUNNING_TASKS: parseInteger({ default: 2 }),
EXCEEDED_MAX_CONCURRENT_RUNNING_TASKS_DELAY_MS: parseInteger({ default: 1000 }),
};
```

2. 작업별 실행 주기

작업	기본 간격	설명
Market Updater	30초	시장 데이터 업데이트 (가격, 거래량, 미결제약정)
PnL Ticks	30초	손익 틱 생성 (실제 업데이트는 1시간마다)
Delete Zero Price Levels	2분	주문서에서 0 수량 가격 레벨 정리
Remove Expired Orders	2분	만료된 주문 제거
Cancel Stale Orders	30초	오래된 주문 취소
Orderbook Instrumentation	5초	주문서 메트릭 수집
Trading Rewards Aggregation	30초	거래 보상 집계 (일간/주간/월간)
Compliance Data Update	5분	컴플라이언스 데이터 업데이트

| Fast Sync Snapshots | 4시간 | 빠른 동기화용 스냅샷 생성 |

| Delete Old Snapshots | 1일 | 오래된 스냅샷 삭제 |

3. 시작 지연 및 지터 시스템

```
function generateRandomStartDelayMs(loopIntervalMs: number): number {
// 최대 지연 시간 계산
const maxDelayFromInterval = loopIntervalMs * config.MAX_START_DELAY_FRACTION_OF
const maxDelay = Math.min(config.MAX_START_DELAY_MS, maxDelayFromInterval);
// 기본 지연 시간
const baseDelay = Math.random() * maxDelay;
// 지터 추가
const jitter = Math.random() * (baseDelay * config.JITTER_FRACTION_OF_DELAY);
return Math.ceil(baseDelay + jitter);
}

// 여러 인스턴스 간 부하 분산을 위한 지연
if (config.START_DELAY_ENABLED) {
await delay(generateRandomStartDelayMs(loopIntervalMs));
}
```

성능 모니터링 및 메트릭

1. 작업별 메트릭 수집

```
export function loopMetricNames(taskName: string) {
return {
startedStat: `${config.SERVICE_NAME}.loops.${taskName}.started`,
completedStat: `${config.SERVICE_NAME}.loops.${taskName}.completed`,
durationRatioStat: `${config.SERVICE_NAME}.loops.duration_ratio`,
timingStat: `${config.SERVICE_NAME}.loops.${taskName}.timing`,
exceededMaxConcurrentTasksStat: `${config.SERVICE_NAME}.loops.exceeded_max_concu
couldNotAcquireExtendedLockStat: `${config.SERVICE_NAME}.loops.could_not_acquire
};
}


// 작업 실행 시 메트릭 수집
stats.gauge(startedStat, 1);
stats.timing(timingStat, loopDuration);
stats.gauge(durationRatioStat, loopDuration / loopIntervalMs, { taskName });
stats.gauge(completedStat, success ? 1 : 0);
```

2. 에러 처리 및 복구

```
try {
await loopTask();
stats.gauge(completedStat, 1);
```

```
} catch (error) {
stats.gauge(completedStat, 0);
logger.error({
message: 'uncaught error in an individual loop',
error,
taskName,
disableGroupingHash: true, // 에러 그룹화 방지
});
}
```

3. 리소스 사용량 모니터링

```
// 동시 실행 작업 수 추적
let numRunningTasks = 0;

// 작업 시작 시
numRunningTasks += 1;
stats.gauge(`${config.SERVICE_NAME}.concurrent_tasks`, numRunningTasks);

// 작업 완료 시
numRunningTasks -= 1;

// 최대 동시 실행 제한 확인
```

```
if (numRunningTasks > config.MAX_CONCURRENT_RUNNING_TASKS) {

stats.increment(exceededMaxConcurrentTasksStat, { taskName });

await delay(config.EXCEEDED_MAX_CONCURRENT_RUNNING_TASKS_DELAY_MS);

continue;

}
```

결론

Roundtable 서비스는 dYdX v4 Indexer의 백그라운드 작업을 체계적으로 관리하는 핵심 컴포넌트로, 분산 환경에서의 안전한 작업 스케줄링, 효율적인 데이터 처리, 그리고 포괄적인 모니터링을 제공합니다. Redis 기반의 분산 잠금 시스템과 지능적인 부하 분산을 통해 높은 가용성과 안정성을 달성하며, 다양한 배치 작업들을 통해 시스템의 데이터 일관성과 성능을 유지하는 중요한 역할을 수행합니다.